

Blood Bank Unit Allocation System

Design Document

1. Terminology & Formulae

- **unit:** a single blood unit in the bank's inventory. Represented as a dict: {"id": str, "group": str, "expiry": str, "status": str}. status is one of "Available", "Issued", or "Wasted".
- **blood group:** one of the eight fixed groups — "A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-".
- **stock:** a list of unit dicts — the bank's full inventory at any moment, expired or issued units included (their status field says which).
- **request:** a hospital's ask for units. Represented as a dict: {"id": str, "hospital": str, "hospital_type": str, "group": str, "quantity": int, "urgency": str, "status": str}. hospital_type is "Private" or "Govt"; urgency is "High"/"Mid"/"Low"; status starts "Pending" and becomes "Fulfilled" or "Rejected".
- **compatibility:** the fixed FR 3 rule for which requested groups a unit of a given stock group may be issued to (e.g. a unit of O- can be issued to any requested group; a unit of AB+ can only be issued to AB+).
- **allocation result:** the outcome of deciding one request against current stock — a dict carrying the status plus either the chosen unit ids, per-unit fees and total (Fulfilled), or the compatible-unit shortfall (Rejected).
- **plan:** the proposed, not-yet-committed output of Generate allocation plan: {"allocations": [...], "proposed_revenue": int}. Nothing in stock or in the requests changes until Confirm & issue plan commits it.
- **shelf-life window:** staff always type the expiry date themselves — there is no default or auto-fill. What's new is validation: an entered date is only accepted if it falls within 35 days of today (inclusive). A date further out than that is rejected the same way as a past date. 35 days is used because it lines up with the shelf life generally cited for stored whole blood and red cell units, so the app can't be made to accept blood dated further into the future than it would realistically still be usable.
- **fee formulae (FR 3):**
 - $\text{per_unit_fee} = 500 + \text{urgency_adjustment} + \text{group_adjustment} - \text{hospital_discount}$
 - urgency_adjustment: High = 300, Mid = 100, Low = 0
 - group_adjustment: Rh-negative (A-, B-, AB-, O-) = 150, Rh-positive (A+, B+, AB+, O+) = 0
 - hospital_discount: Private = 0, Govt = 150
 - $\text{bill_total} = (\text{sum of per_unit_fee over all issued units}) + 100$ (transportation, once per fulfilled request)
- **session totals:** the running counts of Fulfilled/Rejected requests and total revenue (FR 7), plus the running list of wasted units and value lost (FR 8), carried across the whole session.

2. Program Flow

1. Start empty. Before anything happens, the program sets everything to its starting point: no stock yet, no requests waiting, no plan made, and all totals (revenue, counts) at zero.
2. Check for expired blood. Any unit whose date has already passed gets marked as wasted and removed from what's available.
3. Show the stock table and menu. Print what's currently in stock, then the 7 numbered options.
4. Ask the staff to choose an option. They type a number.
5. If it's not a valid number (1–7), show an error and ask again — go back to step 3.
6. Otherwise, run whichever action they picked: Add stock, Submit a request, Generate an allocation plan, Confirm and issue a plan, View progress report, View expired/wasted report, or Exit.
7. Unless they chose Exit, go back to step 2 and repeat the whole thing.

8. If they chose Exit, print "Goodbye" and the program stops.

4. Data Structures

Every data structure the program passes around, where it's used, and what it's for:

Data Structure	Used In	What It Does
Hash map	Stock storage	A unit — one blood item's id, blood group, expiry date, and status.
List of hash maps	Inventory	Stock — the bank's full collection of units at any moment.
Hash map	Request queue	A request — one hospital's ask for blood, its info and status.
Hash map	Allocation logic	The compatibility table — maps each donor group to the groups it can supply.
Hash map	Billing & planning	An allocation result — outcome of matching one request against stock.
Hash map	Plan review	A plan — the proposed batch of allocation results and total revenue.
Hash map	Session tracking	The session log — running totals for the session.
List of hash maps	Request queue	Pending requests — waiting Mid/Low requests.
Hash map	Plan state	The current plan — most recently generated plan.

5. Function Signatures

blood_bank_logic.py — functional core (all pure):

Typed header line	What it's for
<code>def new_session_log() -> dict:</code>	Return a fresh totals dict: {"fulfilled": 0, "rejected": 0, "revenue": 0, "wasted_units": [], "wasted_value": 0}.
<code>def is_valid_group(group: str) -> bool:</code>	True if group is one of the eight valid blood groups.
<code>def is_valid_quantity(quantity) -> bool:</code>	True if quantity is a positive whole number.
<code>def is_valid_urgency(urgency: str) -> bool:</code>	True if urgency is "High", "Mid", or "Low".
<code>def is_valid_hospital_type(hospital_type: str) -> bool:</code>	True if hospital_type is "Private" or "Govt".
<code>def is_expired(expiry: str, today: str) -> bool:</code>	True if expiry is earlier than today.
<code>def is_within_shelf_life(expiry: str, today: str) -> bool:</code>	New. True if expiry is no more than 35 days after today (a date exactly 35 days out counts as within range). Used alongside <code>is_expired</code> to validate an entered date on the far end, not just the near end.
<code>def add_unit(stock: list, unit_id: str, group: str, expiry: str, today: str) -> list:</code>	Return a NEW stock list with one more "Available" unit appended, or unchanged if expiry is already past OR more than 35 days from today (checked via <code>is_expired</code> and <code>is_within_shelf_life</code>). Both rejection cases are treated identically by the caller — unit not added.
<code>def expire_units(stock: list, today: str) -> tuple:</code>	Return (new_stock, newly_wasted_units): a NEW stock list with any Available unit past expiry marked Wasted, plus the units just wasted.
<code>def compatible_available_units(stock: list, request_group: str) -> list:</code>	Return every Available unit whose group is compatible with request_group per the FR 3 rule, sorted soonest-expiry first, ties broken by earlier unit id.
<code>def per_unit_fee(group: str, urgency: str, hospital_type: str) -> int:</code>	Compute one issued unit's fee from the FR 3 formula.
<code>def allocate_request(stock: list, request: dict) -> dict:</code>	Decide one request against current stock: Fulfilled (chosen unit ids, fees, total) or Rejected (compatible-count). Never changes stock.
<code>def issue_units(stock: list, unit_ids: list) -> list:</code>	Return a NEW stock list with the given unit ids marked "Issued".
<code>def sort_for_plan(requests: list) -> list:</code>	Return the pending Mid/Low requests reordered: Mid before Low, original received order preserved within each.

Typed header line	What it's for
<code>def generate_plan(stock: list, requests: list) -> dict:</code>	Simulate allocate_request over every request from sort_for_plan in order; return {"allocations": [...], "proposed_revenue": int}. Never changes the real stock.
<code>def is_plan_valid(stock: list, plan: dict) -> bool:</code>	True if every unit id proposed in plan's Fulfilled allocations is still status "Available" in stock; False if any has expired or been issued since the plan was generated.
<code>def apply_plan(stock: list, plan: dict, session_log: dict) -> tuple:</code>	Commit a plan: return (new_stock, new_session_log) with every Fulfilled allocation's units Issued and its fee added to revenue.
<code>def wasted_report(session_log: dict) -> dict:</code>	Return the count and total value of all units logged as wasted so far.
<code>def progress_report(session_log: dict) -> dict:</code>	Return total handled, fulfilled count, rejected count, and revenue so far.

main.py — imperative shell (all input/output and the changing state):

Typed header line	What it's for
<code>def render_stock_table(stock: list) -> str:</code>	Build the eight-row stock table (group, available count, nearest expiry) as printable text.
<code>def render_menu() -> str:</code>	Build the seven-option menu as printable text.
<code>def read_add_stock_input() -> list:</code>	Prompt for one or more "group expiry" pairs, comma-separated; return them as a list of dicts, each {"group": str, "expiry": str}. The expiry is always required from staff — there is no shorthand for omitting it.
<code>def read_request_input() -> dict:</code>	Prompt for hospital name, hospital type, blood group, quantity, and urgency; return them as a raw request dict.
<code>def print_bill(allocation: dict) -> None:</code>	Print the itemized bill for a Fulfilled allocation, or the rejection message for a Rejected one.
<code>def print_plan(plan: dict) -> None:</code>	Print every request's proposed outcome in a plan, then the proposed total revenue.
<code>def print_progress_report(report: dict) -> None:</code>	Print the FR 7 progress report.
<code>def print_wasted_report(report: dict, stock: list) -> None:</code>	Print the FR 8 expired/wasted report.
<code>def main() -> None:</code>	Run the session loop: expire units, show the stock table and menu, read a choice, dispatch to the matching action, repeat until Exit.

6. Function-Level Algorithm

blood_bank_logic.py

new_session_log()

- Return {"fulfilled": 0, "rejected": 0, "revenue": 0, "wasted_units": [], "wasted_value": 0}.

is_valid_group(group)

- Return True if group is in the fixed set of eight groups, else False.

is_valid_quantity(quantity)

- Return True if quantity can be read as a whole number and is greater than zero, else False.

is_valid_urgency(urgency)

- Return True if urgency is exactly "High", "Mid", or "Low", else False.

is_valid_hospital_type(hospital_type)

- Return True if hospital_type is exactly "Private" or "Govt", else False.

is_expired(expiry, today)

- Return True if expiry is strictly before today, else False.

is_within_shelf_life(expiry, today) — *new*

- Parse today (YYYY-MM-DD), add 35 days to it to get the cutoff date.
- Return True if expiry is on or before that cutoff date (so exactly 35 days out is still True), else False.
- This is the single place the 35-day figure lives — nothing else hardcodes it, so the window can change later without touching add_unit.

add_unit(stock, unit_id, group, expiry, today)

- If is_expired(expiry, today) is True, return stock unchanged — the caller reports it as not added (already in the past).
- If is_within_shelf_life(expiry, today) is False, also return stock unchanged — the caller reports it as not added (too far in the future). Both checks gate on the same expiry value; a date only gets through if it's on or after today AND no more than 35 days after today.
- Otherwise, copy stock, append a new unit dict {"id": unit_id, "group": group, "expiry": expiry, "status": "Available"}, and return the copy.

expire_units(stock, today)

- Copy stock. For every unit whose status is "Available" and is_expired(its expiry, today) is True, set its status to "Wasted" in the copy and add it to a wasted list.
- Return (new_stock, wasted_list).

compatible_available_units(stock, request_group)

- Filter stock to units with status "Available" whose group is compatible with request_group, checked directly against the fixed FR 3 compatibility table (e.g. O- matches every group; AB+ matches only AB+) — there is no separate can_issue helper.
- Sort the result by expiry ascending (soonest first); for units sharing the same expiry, break ties by unit id ascending. Return it.

per_unit_fee(group, urgency, hospital_type)

- Start at 500.
- Add 300, 100, or 0 depending on urgency being High, Mid, or Low.
- Add 150 if group is Rh-negative, else add 0.
- Subtract 150 if hospital_type is "Govt", else subtract 0.
- Return the result.

allocate_request(stock, request)

- Call compatible_available_units(stock, request's group).
- If fewer of them exist than request's quantity: return {"status": "Rejected", "available": , "needed": }. This same branch covers both a total shortage (zero compatible units) and a partial shortage (some, but not enough).
- Otherwise take the first quantity units (soonest-expiry first); compute per_unit_fee for each; total = sum of fees + 100 transportation.
- Return {"status": "Fulfilled", "unit_ids": [...], "fees": [...], "total": total}.

issue_units(stock, unit_ids)

- Copy stock; for every unit whose id is in unit_ids, set its status to "Issued".
- Return the copy.

sort_for_plan(requests)

- Filter requests to only those still "Pending" with urgency "Mid" or "Low".
- Return them ordered: all Mid requests first (in the order received), then all Low requests (in the order received).

generate_plan(stock, requests)

- Start with a working copy of stock, an empty allocations list, and proposed_revenue = 0.
- For each request from sort_for_plan(requests), in order, call allocate_request on the working copy.
- If Fulfilled: append the result to allocations; mark its unit_ids Issued in the working copy only (so the next request in the plan cannot reuse them); add its total to proposed_revenue.
- If Rejected: append the result to allocations unchanged.
- Return {"allocations": allocations, "proposed_revenue": proposed_revenue}. The real stock passed in is never modified.

is_plan_valid(stock, plan)

- For each Fulfilled allocation in plan's allocations, check that every one of its unit_ids is still present in stock with status "Available" (i.e. nothing it proposed has expired or been issued elsewhere since the plan was generated).
- Return True only if this holds for every unit id in every Fulfilled allocation; otherwise return False.

apply_plan(stock, plan, session_log)

- Assumes the caller has already confirmed is_plan_valid(stock, plan) is True; apply_plan itself does not re-check.
- Copy stock and session_log.
- For each Fulfilled allocation in plan: call issue_units to mark its unit_ids Issued on the stock copy; add its total to revenue; increment fulfilled count.
- For each Rejected allocation in plan: increment rejected count.
- Return (new_stock, new_session_log).

wasted_report(session_log)

- Return {"count": len(wasted_units), "value": wasted_value} read from session_log.

progress_report(session_log)

- Return {"handled": fulfilled + rejected, "fulfilled": fulfilled, "rejected": rejected, "revenue": revenue}.

main.py

render_stock_table(stock)

- For each of the eight groups, count Available units and find the soonest expiry among them (or “-” if none).
- Build the table text with those rows and return it.

render_menu()

- Return the seven-line numbered menu text.

read_add_stock_input()

- Prompt once for a comma-separated list of “group expiry” pairs.
- Split on commas, then split each piece on whitespace into group and expiry; return the list of {"group": str, "expiry": str} dicts.

read_request_input()

- Prompt in turn for hospital name, hospital type, blood group, quantity, and urgency.
- Return them, unvalidated, as a raw request dict for the core to check.

print_bill(allocation)

- If status is "Fulfilled": print the BILL header, one line per unit id/group/expiry/fee, the transportation charge, and the total.
- If "Rejected": print the requested/available/needed counts and the rejection message.

print_plan(plan)

- For each allocation in plan's allocations, print its request and outcome (Fulfilled with units and total, or Rejected with the shortfall).
- Print the proposed_revenue total and the reminder to confirm.

print_progress_report(report)

- Print handled/fulfilled/rejected counts and the revenue so far.

print_wasted_report(report, stock)

- Print each Wasted unit's id, group, and expiry, then the count and total value from report.

main()

- Set stock = [] (built up directly, no separate constructor function), pending_requests = [], current_plan = None, session_log = new_session_log(), and id counters = 1.
- Loop:
- stock, wasted = expire_units(stock, today); if wasted, print them and fold their value into session_log.
- Print render_stock_table(stock) and render_menu(); read the chosen option, safely handling non-numeric input (e.g. “abc”) as invalid rather than crashing.
- If the option is not 1-7: print an error and loop again.
- 1 (Add stock): read_add_stock_input(); for each pair, if is_valid_group is False, print an error naming the invalid blood group and skip that pair, still processing the remaining valid pairs in the same entry; otherwise call add_unit and print what was added, or the appropriate rejection message if the date had already passed (is_expired) or was more than 35 days out (is_within_shelf_life is False) — both cases mean the unit is not added, and the message names which limit was hit.

- 2 (Submit request): `read_request_input()`; validate group/quantity/urgency/hospital type, printing the matching error and stopping if any fail; if urgency is High, call `allocate_request` immediately and `print_bill`, then update `session_log` and stock via `issue_units`; otherwise append the request (status "Pending") to `pending_requests`.
- 3 (Generate allocation plan): if `pending_requests` has no Mid/Low entries, print a "nothing to generate" message; otherwise call `generate_plan`, store the result as `current_plan`, and `print_plan(current_plan)`.
- 4 (Confirm & issue plan): if `current_plan` is None, print an error; otherwise call `is_plan_valid(stock, current_plan)` — if False, print an error that the plan is stale and clear `current_plan`, requiring staff to regenerate before confirming, with no state change; if True, call `apply_plan`, replace stock and `session_log` with the results, remove the confirmed requests from `pending_requests`, clear `current_plan`, and print the confirmation and total revenue.
- 5 (View progress report): `print_progress_report(progress_report(session_log))`.
- 6 (View expired/wasted report): `print_wasted_report(wasted_report(session_log), stock)`.
- 7 (Exit): print "Goodbye" and stop the loop.